

题目一：存储管理

题目种类：内核代码填充

推荐知识点：存储管理的基本原理、文件存储组织、记录存储组织、缓冲区管理

题目描述：

在课程提供的代码框架的基础上，提供相关接口的实现，完成指定功能。本题目包含若干子任务，每个任务对应不同的测试点，只有通过相应测试点才可以获得相应的分数。

提示：

(1) 测试代码会调用指定接口来进行测试，学生不能修改已有的接口，也不能删除已有的数据结构及数据结构中的变量，但是可以增添新的接口、数据结构、变量。

(2) 课程提供了项目结构文档来帮助学生理解代码框架，同时在代码注释中，也对数据结构及接口进行了说明，学生可以通过阅读代码注释来辅助理解代码框架。

1、磁盘管理器

在本任务中，学生需要实现磁盘管理器 `DiskManager` 的相关接口，磁盘管理器负责文件操作、读写页面等。在完成本任务之前，学生需要阅读项目结构文档中磁盘管理器的相关说明，以及代码框架中 `src/errors.h`、`src/storage/disk_manager.h`、`src/storage/disk_manager.cpp`、`src/common/config.h` 文件。

学生需要实现以下接口：

(1) `void DiskManager::create_file(const std::string &path);`

该接口的参数 `path` 为文件名，该接口的功能是创建文件，其文件名为 `path` 参数指定的文件名。

(2) `void DiskManager::open_file(const std::string &path);`

该接口的参数 `path` 为文件名，该接口的功能是打开文件名参数 `path` 指定的文件。

(3) `void DiskManager::close_file(const std::string &path);`

该接口的参数 `path` 为文件名，该接口的功能是关闭文件名参数 `path` 指定的文件。

(4) `void DiskManager::destroy_file(const std::string &path);`

该接口的参数 `path` 为文件名，该接口的功能是删除文件名参数 `path` 指定的文件。

(5) `void DiskManager::write_page(int fd, page_id_t page_no, const char *offset, int num_bytes);`

该接口负责在文件的指定页面写入指定长度的数据，该接口从指定页面的起始位置开始写入数据。

(6) `void DiskManager::read_page(int fd, page_id_t page_no, char *offset, int num_bytes);`

该接口需要从文件的指定页面读取指定长度的数据，该接口从指定页面的起始位置开始读取数据。

2、缓冲池管理器

在本任务中，学生需要实现缓冲池管理器 `BufferPoolManager` 和缓冲池替换策略 `Replacer` 相关的接口，缓冲池管理器负责管理缓冲池中的页面在内外存的交换，缓冲池替换策略主要负责缓冲区页面的淘汰和查找。在完成本任务之前，学生需要首先阅读项目结构文档中缓冲池管理器的相关说明，以及代码框架中 `src/storage` 和 `src/replacer` 文件夹下的代码文件。

对于缓冲池替换策略 `Replacer`，学生需要实现一个 `Replacer` 的子类 `LRUReplacer`，`LRUReplacer` 实现了缓冲池页面替换的 LRU 策略，需要实现的接口如下：

(1) `bool LRUReplacer::victim(frame_id_t* frame_id);`

该接口的功能是选择并淘汰缓冲池中一个页面。如果成功找到要淘汰的页面，则函数返回值为 `true`；否则，返回值为 `false`。被淘汰的页面所在的帧由参数 `frame_id` 返回。

(2) `void LRURemplacer::pin(frame_id_t frame_id);`

该接口的功能是固定住一个帧中的页面，代表该页面正在使用，不可被换出，参数 `frame_id` 指定了帧的编号。

(3) `void LRURemplacer::unpin(frame_id_t frame_id);`

该接口的功能是取消固定一个帧中的页面，当该页面使用完毕时调用 `unpin` 函数取消对该页面的固定，参数 `frame_id` 指定了帧的编号。

对于缓冲池管理器 `BufferPoolManager`，学生需要管理缓冲池中的页面，并对缓冲池进行并发控制，需要实现的接口如下：

(1) `Page *BufferPoolManager::new_page(PageId *page_id);`

该成员函数用于在缓冲池中申请创建一个新页面。如果创建新页面成功，则返回指向该页面的指针，同时通过参数 `page_id` 返回新建页面的编号。

(2) `Page *BufferPoolManager::fetch_page(PageId page_id);`

该成员函数用于获取缓冲池中的指定页面。待获取页面的编号由参数 `page_id` 给出。

(3) `bool BufferPoolManager::find_victim_page(frame_id_t *frame_id);`

当缓冲池中沒有可用的空闲帧时，该成员函数用于寻找需要淘汰的页面。如果成功找到要淘汰的页面，则函数返回值为 `true`；否则，返回值为 `false`。被淘汰的页面所在的帧由参数 `frame_id` 返回。在实现这个成员函数时，需要调用 `LRURemplacer::victim` 函数。

(4) `void BufferPoolManager::update_page(Page *page, PageId new_page_id, frame_id_t new_frame_id);`

当缓冲池想要把某个帧中的页面置换为新页面或者删除该帧中的页面时，会调用 `update_page` 函数，该函数把指定帧中的原有页面刷入磁盘中，并将新页面和该帧建立映射，即更新页表。

(5) `bool BufferPoolManager::unpin_page(PageId page_id, bool is_dirty);`

当某个操作完成某个页面的使用之后，需要调用该函数将取消该操作对该页面的固定。当所有操作都完成该页面的使用之后，需要在 `Replacer` 中调用 `Unpin` 函数取消该页面的固定。

(6) `bool BufferPoolManager::delete_page(PageId page_id);`

用于在缓冲池中删除指定页面，同时将该页面所在的帧置为空闲帧。如果当前页面正在被某个操作使用，则该页面不能被删除。

(7) `bool BufferPoolManager::flush_page(PageId page_id);`

用于强制刷新缓冲池中的指定页面到磁盘上，无论该页是否正在被使用，或者是否为脏页，都需要把该页面的数据刷入磁盘中。

(8) `void BufferPoolManager::flush_all_pages(int fd);`

用于将指定文件中的存在于缓冲池的所有页面都刷新到磁盘。

提示：在缓冲池中，需要淘汰某个脏页时，需要将脏页写入磁盘。

3、记录管理器

在本任务中，学生需要填充记录管理器涉及的 `RmFileHandle` 类和 `RmScan` 类，`RmFileHandle` 类负责对表的记录进行操作，`RmScan` 类用于遍历表文件中存放的记录。

对于 `RmFileHandle` 类。在完成本文任务之前，学生需要阅读项目结构文档中记录管理器的相关说明，以及代码框架中 `src/record` 文件夹下的代码文件。

学生需要实现的接口如下：

(1) `std::unique_ptr<RmRecord> RmFileHandle::get_record(const Rid &rid, Context *context) const;`

该函数用于获取表中某一条指定位置的记录，每条记录由 Rid 来进行唯一标识。

(2) `Rid RmFileHandle::insert_record(char *buf, Context *context);`

该函数负责向表中插入一条新记录，在该函数中未指定记录的插入位置，学生需要选择一个空闲位置插入记录并同步更新表的元数据信息。

(3) `void RmFileHandle::insert_record(const Rid &rid, char *buf);`

该函数负责向表中的指定位置插入一条记录，该函数主要用于事务的回滚和系统故障恢复。

(4) `void RmFileHandle::delete_record(const Rid &rid, Context *context);`

该函数负责删除表中指定位置的记录。

(5) `void RmFileHandle::update_record(const Rid &rid, char *buf, Context *context);`

该函数负责把表中指定位置的记录更新为新的值。

对于 RmScan 类，学生需要实现的接口如下：

(1) `RmScan(const RmFileHandle *file_handle);`

该函数为 RmScan 类的构造函数，需要初始化相关成员变量。

(2) `void RmScan::next() override;`

该函数负责找到表文件中下一个存放了合法记录的位置。

(3) `bool RmScan::is_end() const override;`

该函数负责判断是否已经扫描到文件的末尾位置。

测试示例：

unit_test.cpp 中提供了参考测试示例，最终测试包括但不限于 unit_test.cpp 中提供的测试。

题目二：查询执行

前置题目：题目一（存储管理）

题目种类：基础功能

推荐知识点：逻辑查询优化、查询解析、基本算子实现、查询执行框架、元数据存储组织

代码框架：<https://gitlab.eduxiji.net/csc-db/db2023/-/tree/main/rmdb>

题目描述：

在实现题目一功能的基础上增加查询执行模块，使得系统支持通过测试需要的元数据管理、DDL 语句、DQL 语句、DML 语句。

提示：

（1）本题目提供了空缺的样例代码，参赛队伍可以选择进行补全，也可以选择自行构建。如果选择补全空缺的样例代码，需要实现“元数据管理与 DDL 语句”和“DQL 语句和 DML 语句”。

（2）本题目通过 sql 进行测试，分为五个测试点，它们依次是“尝试建表”、“单表插入与条件查询”、“单表更新与条件查询”、“单表删除与条件查询”、“连接查询”。

（3）对于所有不合法的 sql 语句，都需要输出 **failure**。（包括语法错误、删除不存在的表、创建已经存在的表、where 条件中出现不存在的字段等）

（4）注意浮点数的精度问题，在最后一个测试点 `basic_query_test5` 中涉及到了浮点数精度的测试。（每个测试点的测试知识点参考测试说明文档）

1、元数据管理与 DDL 语句

本任务要求参赛队伍对数据库的元数据进行管理，并实现基本的 DDL 语句，包括 `create table` 和 `drop table` 两种语句。大赛提供的框架中，与元数据管理和 DDL 语句相关的代码文件位于 `src/system` 文件夹下，代码框架中提供了 `create table` 语句的实现方式，参赛队伍需要补充完善 `sm_manager.cpp` 文件中未实现的函数（`index` 相关的函数除外）。

2、DQL 语句和 DML 语句

DML 语句要求实现基本的增删改，即 `insert`、`delete`、`update` 语句。DQL 语句要求实现 `select` 语句。在完成本任务之前，参赛队伍需要首先阅读项目结构文档中查询解析、查询优化、查询执行的相关说明，以及代码框架中 `src/analyze`、`src/optimizer`、`src/execution` 文件夹下的代码文件，参赛队伍需要实现代码框架中未提供的功能，包括语义检查、查询执行计划的生成、执行算子等。语义检查包括但不限于创建已有的表、删除不存在的表等。

在大赛提供的框架中，`analyze` 模块提供了 `insert`、`select`、`delete` 语句的相关代码，把 `parser` 生成的抽象语法树转化成了查询计划生成所需要的 `Query` 列表，并进行了部分语义检查，参赛队伍需要补充完善 `update` 语句的相关代码。`optimizer` 模块提供了查询计划的生成，但并未对查询计划进行逻辑优化和物理优化。`execution` 模块中提供了 `insert` 算子的完整实现，参赛队伍需要实现扫描算子、连接算子、投影算子、`update` 算子和 `delete` 算子。

本题目通过 SQL 来进行测试，因此参赛队伍可以对代码进行重构。

测试示例：

测试点 1: 尝试建表 (2 分)

测试示例:

```
create table t1(id int,name char(4));
```

```
show tables;
```

```
create table t2(id int);
```

```
show tables;
```

```
drop table t1;
```

```
show tables;
```

```
drop table t2;
```

```
show tables;
```

期待输出:

```
| Tables |
```

```
| t1 |
```

```
| Tables |
```

```
| t1 |
```

```
| t2 |
```

```
| Tables |
```

```
| t2 |
```

```
| Tables |
```

测试点 2: 单表插入与条件查询 (2 分)

测试示例:

```
create table grade (name char(4),id int,score float);
```

```
insert into grade values ('Data', 1, 90.5);
```

```
insert into grade values ('Data', 2, 95.0);
```

```
insert into grade values ('Calc', 2, 92.0);
```

```
insert into grade values ('Calc', 1, 88.5);
```

```
select * from grade;
```

```
select score,name,id from grade where score > 90;
```

```
select id from grade where name = 'Data';
```

```
select name from grade where id = 2 and score > 90;
```

期待输出:

```
| name | id | score |
```

```
| Data | 1 | 90.500000 |
```

```
| Data | 2 | 95.000000 |
```

```
| Calc | 2 | 92.000000 |
```

```
| Calc | 1 | 88.500000 |
```

```
| score | name | id |
```

```
| 90.500000 | Data | 1 |
```

```
| 95.000000 | Data | 2 |
```

```
| 92.000000 | Calc | 2 |
```

```
| id |
```

```
| 1 |
```

```
| 2 |  
| name |  
| Data |  
| Calc |
```

测试点 3: 单表更新与条件查询 (2 分)

测试示例:

```
create table grade (name char(4),id int,score float);  
insert into grade values ('Data', 1, 90.5);  
insert into grade values ('Data', 2, 95.0);  
insert into grade values ('Calc', 2, 92.0);  
insert into grade values ('Calc', 1, 88.5);  
select * from grade;  
update grade set score = 99.0 where name = 'Calc' ;  
select * from grade;  
update grade set name = 'test' where name > 'A';  
select * from grade;  
update grade set name = 'test' ,id = -1,score = 0 where name = 'test' and score > 90;  
select * from grade;
```

期待输出:

```
| name | id | score |  
| Data | 1 | 90.500000 |  
| Data | 2 | 95.000000 |  
| Calc | 2 | 92.000000 |  
| Calc | 1 | 88.500000 |  
| name | id | score |  
| Data | 1 | 90.500000 |  
| Data | 2 | 95.000000 |  
| Calc | 2 | 99.000000 |  
| Calc | 1 | 99.000000 |  
| name | id | score |  
| test | 1 | 90.500000 |  
| test | 2 | 95.000000 |  
| test | 2 | 99.000000 |  
| test | 1 | 99.000000 |  
| name | id | score |  
| test | -1 | 0.000000 |  
| test | -1 | 0.000000 |  
| test | -1 | 0.000000 |  
| test | -1 | 0.000000 |
```

测试点 4: 单表删除与条件查询 (2 分)

测试示例:

```
create table grade (name char(4),id int,score float);
```

```
insert into grade values ('Data', 1, 90.5);
```

```
select * from grade;
```

```
delete from grade where score > 90;
```

```
select * from grade;
```

期待输出：

```
| name | id | score|
```

```
| Data | 1 | 90.500000 |
```

```
| name | id | score|
```

测试点 5: 连接查询 (4 分)

```
create table t ( id int , t_name char (3));
```

```
create table d (d_name char(5),id int);
```

```
insert into t values (1,'aaa');
```

```
insert into t values (2,'baa');
```

```
insert into t values (3,'bba');
```

```
insert into d values ('12345',1);
```

```
insert into d values ('23456',2);
```

```
select * from t, d;
```

```
select t.id,t_name,d_name from t,d where t.id = d.id;
```

期待输出：

```
| id | t_name | d_name | id |
```

```
| 1 | aaa | 23456 | 2 |
```

```
| 1 | aaa | 12345 | 1 |
```

```
| 2 | baa | 23456 | 2 |
```

```
| 2 | baa | 12345 | 1 |
```

```
| 3 | bba | 23456 | 2 |
```

```
| 3 | bba | 12345 | 1 |
```

```
| id | t_name | d_name |
```

```
| 1 | aaa | 12345 |
```

```
| 2 | baa | 23456 |
```

测试输出要求：

本题目的输出要求写入数据库文件夹下的 `output.txt` 文件中，例如测试数据库名称为 `execution_test_db`，则在测试时使用 `./bin/rmdb execution_test_db` 命令来启动服务端，对应输出应写入 `buid/execution_test_db/output.txt` 文件中。

题目三：BIGINT 类型

前置题目：题目二（查询执行）

题目种类：基础功能

推荐知识点：记录存储组织、查询处理

代码框架：<https://gitlab.eduxiji.net/csc-db/db2023/-/tree/main/rmdb>

题目描述：

BIGINT 是整数类型，用来存储极大整数值，MySQL 中 BIGINT 大小为 8 字节，有符号整数存储范围为 $(-9,223,372,036,854,775,808 \sim 9,223,372,036,854,775,807)$ 。目前数据库只支持有符号 INT 字段，大小为 4 字节。你需要实现有符号 BIGINT 字段，存储范围同 MySQL 的有符号整数 BIGINT 类型，且需支持在该字段上的增删改查。

提示：功能题目对代码框架没有限制，参赛队伍可以修改、增添、删除数据结构及接口，也可以对框架进行重构。

测试示例：

```
CREATE TABLE t(bid bigint,sid int);
INSERT INTO t VALUES(372036854775807,233421);
INSERT INTO t VALUES(-922337203685477580,124332);
SELECT * FROM t;
INSERT INTO t VALUES(9223372036854775809,12345);
SELECT * FROM t;
```

期待输出：

```
| bid | sid |
| 372036854775807 | 233421 |
| -922337203685477580 | 124332 |
failure
| bid | sid |
| 372036854775807 | 233421 |
| -922337203685477580 | 124332 |
```

测试输出要求：

本题目的输出要求写入数据库文件夹下的 output.txt 文件中，例如测试数据库名称为 bigint_test_db，则在测试时使用 ./bin/rmdb bigint_test_db 命令来启动服务端，对应输出应写入 buid/bigint_test_db/output.txt 文件中。

题目四：时间类型

前置题目：题目二（查询执行）

题目种类：基础功能

推荐知识点：记录存储组织、查询处理

代码框架：<https://gitlab.eduxiji.net/csc-db/db2023/-/tree/main/rmdb>

题目描述：

本题目要求实现 DATETIME 时间类型，DATETIME 类型大小为 8 字节，格式为'YYYY-MM-DD HH:MM:SS'，最小值为'1000-01-01 00:00:00'，最大值为'9999-12-31 23:59:59'，参赛队伍需要判断输入值的合法性（非法输入包括但不限于出现负值、月的值小于 1 或大于 12，日的值小于 1 或大于 31、2 月的天数等于 30、时针数大于 23、分针数大于 59、秒针数大于 59、字段长度不匹配）

提示：功能题目对代码框架没有限制，参赛队伍可以修改、增添、删除数据结构及接口，也可以对框架进行重构。

测试示例：

测试点 1：建表时创建时间类型的属性，并在该字段上进行增删改查

```
create table t(id int , time datetime);
insert into t values(1, '2023-05-18 09:12:19');
insert into t values(2, '2023-05-30 12:34:32');
select * from t;
delete from t where time = '2023-05-30 12:34:32';
update t set id = 2023 where time = '2023-05-18 09:12:19';
select * from t;
```

测试点 2：对输入的合法性进行判断

```
create table t(time datetime, temperature float)
insert into t values('1999-07-07 12:30:00' , 36.0);
select * from t;
insert into t values('1999-13-07 12:30:00' , 36.0);
insert into t values('1999-1-07 12:30:00' , 36.0);
insert into t values('1999-00-07 12:30:00' , 36.0);
insert into t values('1999-07-00 12:30:00' , 36.0);
insert into t values('0001-07-10 12:30:00' , 36.0);
insert into t values('1999-02-30 12:30:00' , 36.0);
insert into t values('1999-02-28 12:30:61' , 36.0);
select * from t;
```

期待输出：

测试点 1：

```
| id | time |
| 1 | 2023-05-18 09:12:19 |
| 2 | 2023-05-30 12:34:32 |
| id | time |
| 2023 | 2023-05-18 09:12:19 |
```

测试点 2:

```
| time | temperature |  
| 1999-07-07 12:30:00 | 36.000000 |  
failure  
failure  
failure  
failure  
failure  
failure  
failure  
| time | temperature |  
| 1999-07-07 12:30:00 | 36.000000 |
```

测试输出要求:

本题目的输出要求写入数据库文件夹下的 `output.txt` 文件中，例如测试数据库名称为 `datetime_test_db`，则在测试时使用 `./bin/rmdb datetime_test_db` 命令来启动服务端，对应输出应写入 `buid/datetime_test_db/output.txt` 文件中。

题目五：唯一索引

前置题目：题目二（查询执行）

题目种类：基础功能

推荐知识点：B+树索引

代码框架：<https://gitlab.eduxiji.net/csc-db/db2023/-/tree/main/rmdb>

题目描述：

在现有系统的基础上增添索引功能，该索引为唯一索引，要求系统能够支持创建索引、删除索引、展示某个表上的索引、单点查询和范围查询，实现索引与基表的同步。

如果一个表上的某个字段建有唯一索引，则该表中任意两个记录中该字段的值不相同。

代码框架 RMDB 中提供了 B+树的实现逻辑，参赛队伍可以阅读 `src/index` 的相关代码，补充完善 B+树索引，并完成 `src/system` 中 `create/drop index` 两个函数。

提示：

（1）功能题目对代码框架没有限制，参赛队伍可以修改、增添、删除数据结构及接口，也可以对框架进行重构。

（2）磁盘数据库中常用的索引为 B+树索引，推荐使用 B+树来实现索引功能，参赛队伍也可以使用其他类型的索引，但需要能够支持题目中要求的功能。

（3）对于测试点 2、3，即“索引的查询”和“维护索引的插入、删除、更新”，会进行单独的是否真正使用了索引的测试——对于数千条查询语句，建立索引后的执行时间应该小于建立索引前的执行时间的 70%，才可以视为真正使用了索引的测试。

（4）改题目不要求使用 B+树作为索引，所以对于参赛队伍测试点 2、3 的输出结果不要求行的顺序与期待输出的行的顺序完全一致，但列的顺序要求完全一致。

1、创建、删除、展示索引（3分）

（1）支持单个字段索引的创建和删除；

（2）支持多个字段索引的创建和删除；

（3）查看某个表上的索引信息；

（4）`show index from table_name` 的输出格式为 `| table_name | unique | (column_name,column_name)|`。参考下方测试用例和期待输出。注意输出时，两个字段中间的逗号后面不要加空格。

测试示例：

```
create table warehouse (id int, name char(8));
```

```
create index warehouse (id);
```

```
show index from warehouse;
```

```
create index warehouse (id,name);
```

```
show index from warehouse;
```

```
drop index warehouse (id);
```

```
drop index warehouse (id,name);
```

```
show index from warehouse;
```

期望输出：

```
| warehouse | unique | (id) | // 第一个 show index 的输出
```

```
| warehouse | unique | (id) | // 第二个 show index 的输出
```

```
| warehouse | unique | (id,name) | // 第二个 show index 的输出
// 第三个 show index 没有输出
```

2、索引查询（9分）

在创建索引后，能够使用索引进行单点查询和范围查询。

提示：

在大赛提供的框架中，只有查询条件与索引完全一致，并且是单点查询时，才使用索引来进行查询，你需要对索引匹配规则进行修改，要求使用最左匹配原则。例如，表 A 的(id, name, score)三个属性上有一个联合索引，对于以下几种查询，都需要使用索引来进行查询：

```
l select * from A where id = 1 and name = 'abcd' and score = 99.0;
l select * from A where id = 1 and name = 'abcd' and score > 90.0;
l select * from A where id = 1 and name = 'abcd';
l select * from A where name = 'abcd' and id = 1; // 在进行查询计划生成时，应能够自动对顺序进行调整
l select * from A where id = 1;
l select * from A where id > 1;
```

测试示例：

```
create table warehouse (w_id int, name char(8));
insert into warehouse values (10, 'qweruiop');
insert into warehouse values (534, 'asdfhjk');
insert into warehouse values (100, 'qwerghjk');
insert into warehouse values (500, 'bgtyhnmj');
create index warehouse(w_id);
select * from warehouse where w_id = 10;
select * from warehouse where w_id < 534 and w_id > 100;
drop index warehouse(w_id);
create index warehouse(name);
select * from warehouse where name = 'qweruiop';
select * from warehouse where name > 'qwerghjk';
select * from warehouse where name > 'aszdefgh' and name < 'qweraaaa';
drop index warehouse(name);
create index warehouse(w_id, name);
select * from warehouse where w_id = 100 and name = 'qwerghjk';
select * from warehouse where w_id < 600 and name > 'bztyhnmj';
期待输出：
```

```
| w_id | name |
| 10 | qweruiop |
| w_id | name |
| 500 | bgtyhnmj |
| w_id | name |
| 10 | qweruiop |
| w_id | name |
```

```
| 10 | qweruiop |
| w_id | name |
| 500 | bgtyhnmj |
| w_id | name |
| 100 | qwerghjk |
| w_id | name |
| 10 | qweruiop |
| 100 | qwerghjk |
```

3、索引维护（6分）

在建有索引的表中插入、删除、更新数据时，能够根据表数据的变化同步对表上对索引进行更新，保证索引的正确性。同时，在索引被更新时，需要检查唯一性约束。

测试示例：

```
create table warehouse (w_id int, name char(8));
insert into warehouse values (10 , 'qweruiop');
insert into warehouse values (534, 'asdfhjkl');
select * from warehouse where w_id = 10;
select * from warehouse where w_id < 534 and w_id > 100;
create index warehouse(w_id);
insert into warehouse values (500, 'lastdanc');
update warehouse set w_id = 507 where w_id = 534;
select * from warehouse where w_id = 10;
select * from warehouse where w_id < 534 and w_id > 100;
```

期待输出：

```
| w_id | name |
| 10 | qweruiop |
| w_id | name |
| w_id | name |
| 10 | qweruiop |
| w_id | name |
| 500 | lastdanc |
| 507 | asdfhjkl |
```

测试输出要求：

本题目的输出要求写入数据库文件夹下的 output.txt 文件中，例如测试数据库名称为 index_test_db，则在测试时使用 ./bin/rmdb index_test_db 命令来启动服务端，对应输出应写入 buid/index_test_db/output.txt 文件中。

题目六：聚合函数

前置题目：题目二（查询执行）

题目种类：基础功能

推荐知识点：查询解析、查询处理

代码框架：<https://gitlab.eduxiji.net/csc-db/db2023/-/tree/main/rmdb>

题目描述：

聚合函数对一组值进行计算并返回单一的值，通过使用 SQL 聚合函数，可以确定数值集合的各种统计值

1 COUNT() - 返回行数

1 MAX() - 返回最大值

1 MIN() - 返回最小值

1 SUM() - 返回总和

其中，count、max、min 涉及到 int、float、char 字段，sum 只涉及 int 和 float 两种字段。

提示：功能题目对代码框架没有限制，参赛队伍可以修改、增添、删除数据结构及接口，也可以对框架进行重构。

测试点 1：SUM,浮点数保留 6 位小数,整数不显示小数, as 别名要求与 SQL 一致（1 分）

测试示例：

```
create table aggregate (id int, val float);
insert into aggregate values(1,5.5);
insert into aggregate values(3,4.5);
insert into aggregate values(5,10.0);
select SUM(id) as sum_id from aggregate;
select SUM(val) as sum_val from aggregate;
```

期待输出：

```
| sum_id |
| 9 |
| sum_val |
| 20.000000 |
```

测试点 2：MAX,MIN（2 分）

测试示例：

```
create table aggregate (id int, val float);
insert into aggregate values(1,5.5);
insert into aggregate values(3,4.5);
insert into aggregate values(5,10.0);
select MAX(id) as max_id from aggregate;
select MIN(val) as min_val from aggregate;
```

期待输出：

```
| max_id |
| 5 |
| min_val |
```

| 4.500000 |

测试点 3: COUNT(),COUNT(*) (1 分)

测试示例:

```
create table aggregate (id int,name char(8),val float);
insert into aggregate values (1,'qwertasdf',1.0);
insert into aggregate values (2,'qwertasdf',2.0);
insert into aggregate values (3,'uiophjkl',2.0);
select COUNT(*) as count_row from aggregate;
select COUNT(id) as count_id from aggregate;
select COUNT(name) as count_name from aggregate where val = 2.0;
```

期待输出:

```
| count_row |
| 3 |
| count_id |
| 3 |
| count_name |
| 2 |
```

测试输出要求:

本题目的输出要求写入数据库文件夹下的 output.txt 文件中，例如测试数据库名称为 aggregator_test_db，则在测试时使用 ./bin/rmdb aggregator_test_db 命令来启动服务端，对应输出应写入 build/aggregator_test_db/output.txt 文件中。

题目七：order by 操作符

前置题目：题目二（查询执行）

题目种类：基础功能

推荐知识点：查询解析、查询处理、基本算子实现

代码框架：<https://gitlab.eduxiji.net/csc-db/db2023/-/tree/main/rmdb>

题目描述：

ORDER BY 关键字用于对结果集按照一个列或者多个列进行排序，ORDER BY 关键字默认按照升序对记录进行排序，可以使用 DESC 关键字来指定按照降序排列，使用 ASC 关键字指定按照升序排列。同时，本题目要求支持 LIMIT 关键字，LIMIT 关键字用来限制输出结果集的大小。

提示：

（1）功能题目对代码框架没有限制，参赛队伍可以修改、增添、删除数据结构及接口，也可以对框架进行重构。

（2）参赛队伍可以通过实现 Sort 算子来实现 order by 的功能。

测试示例：

```
create table orders (company char(10), order_number int);
insert into orders values('AAA',12);
insert into orders values('ABB',13);
insert into orders values('ABC',19);
insert into orders values('ACA',1);
SELECT company, order_number FROM orders ORDER BY order_number;
SELECT company, order_number FROM orders ORDER BY company, order_number;
SELECT company, order_number FROM orders ORDER BY company DESC, order_number ASC;
SELECT company, order_number FROM orders ORDER BY order_number ASC LIMIT 2;
```

期待输出：

```
| company | order_number |
| ACA | 1 |
| AAA | 12 |
| ABB | 13 |
| ABC | 19 |
| company | order_number |
| AAA | 12 |
| ABB | 13 |
| ABC | 19 |
| ACA | 1 |
| company | order_number |
| ACA | 1 |
| ABC | 19 |
| ABB | 13 |
| AAA | 12 |
| company | order_number |
| ACA | 1 |
```


测试输出要求：

本题目的输出要求写入数据库文件夹下的 `output.txt` 文件中，例如测试数据库名称为 `aggregator_test_db`，则在测试时使用 `./bin/rmdb aggregator_test_db` 命令来启动服务端，对应输出应写入 `buid/aggregator_test_db/output.txt` 文件中。

题目八：事务控制语句

前置题目：题目五（查询执行）、题目四（唯一索引，部分测试点涉及索引）、题目三（时间类型，部分测试点涉及时间类型）

题目种类：基本功能

推荐知识点：事务的基本概念

代码框架：<https://gitlab.eduxiji.net/csc-db/db2023/-/tree/main/rmdb>

题目描述：

系统需要支持显示开启事务（begin）、提交事务（commit）、回滚事务（abort）三条事务控制语句，显式事务中只包含增删改查四种语句，不包含 DDL 语句。在大赛提供的框架中，单条语句被包装成一个单独的事务进行执行，参赛队伍可以自行更改。在本任务中，不需要考虑并发事务的执行，测试数据中不包含并发事务。

在完成本任务之前，参赛队伍可以首先阅读项目结构文档中事务管理器的相关说明，以及代码框架中 src/transaction 文件夹下的代码文件。

提示：功能题目对代码框架没有限制，参赛队伍可以在原有框架的基础上进行实现，也可以修改、增添、删除数据结构及接口，或者对框架进行重构。

测试示例：

本测试通过包含四个测试点，分别测试有索引和无索引情况下事务的提交与回滚，其中有索引的测试数据中包含时间类型，但不包含不合法的时间类型，测试数据为 TPC-C 中的 NewOrder 事务。测试语句格式如下：

```
create table student (id int, name char(8), score float);
```

```
insert into student values (1, 'xiaohong', 90.0);
```

```
begin;
```

```
insert into student values (2, 'xiaoming', 99.0);
```

```
delete from student where id = 2;
```

```
abort;
```

```
select * from student;
```

期待输出：

```
| id | name | score |
```

```
| 1 | xiaohong | 90.000000 |
```

测试输出要求：

本题目的输出要求写入数据库文件夹下的 output.txt 文件中，例如测试数据库名称为 transaction_test_db，则在测试时使用 ./bin/rmdb transaction_test_db 命令来启动服务端，对应输出应写入 build/transaction_test_db/output.txt 文件中。

题目九：基于死锁预防的可串行化隔离级别

前置题目：题目九（事务控制语句）、题目四（唯一索引）

题目描述：基本功能

推荐知识点：数据异常与隔离级别、可串行化与冲突可串行化、两阶段封锁协议、多粒度封锁、死锁预防

代码框架：<https://gitlab.eduxiji.net/csc-db/db2023/-/tree/main/rmdb>

题目描述：

系统需要支持并发事务，参赛队伍需要实现两阶段封锁算法来保证可串行化隔离级别，对于可能出现的死锁现象，参赛队伍需要实现 no-wait 算法来进行死锁预防。

在完成本任务之前，参赛队伍可以首先阅读项目结构文档中并发控制的相关说明，以及代码框架中 src/transaction 文件夹下的代码文件。

提示：

（1）所有的共享数据都需要考虑并发访问的正确性，例如事务管理器中的 `txn_map` 和索引；

（2）由于存在范围查询，因此，参赛队伍需要考虑幻读数据异常，对于幻读数据异常，可以通过加表级锁来规避数据异常，但是为了实现更好的性能，推荐在索引中使用间隙锁。

测试示例：

本测试通过判断系统是否会出现数据异常来进行可串行化测试，包括脏写、脏读、丢失更新、不可重复读、幻读五种数据异常的测试：

例如，对脏读数据异常进行如下测试：

```
create table concurrency_test (id int, name char(8), score float);
insert into concurrency_test values (1, 'xiaohong', 90.0);
insert into concurrency_test values (2, 'xiaoming', 95.0);
insert into concurrency_test values (3, 'zhanghua', 88.5);
```

事务 1:

```
t1a begin;
t1b update concurrency_test set score = 100.0 where id = 2;
t1c abort;
t1d select * from concurrency_test where id = 2;
```

事务 2:

```
t2a begin;
t2b select * from concurrency_test where id = 2;
t2c commit;
```

操作序列：t1a t2a t1b t2b t1c t1d

测试输出要求：

本题目的测试通过客户端的收到的返回字符串来进行正确性判断，因此参赛队伍不能修改返回给客户端时记录的格式，也就是不能修改 `src/record_printer.h` 文件，格式化的 `select` 语句输出示例如下：

id	name	score
2	xiaoming	95.000000
Total record(s): 1		

对于因为死锁预防策略而需要回滚的事务，参赛队伍需要将“abort\n”返回给客户端，除了上述 select 语句的输出以及事务的回滚信息之外，参赛队伍不应该将多余无用信息返回给客户端。

题目十：系统故障恢复

前置题目：题目十（可串行化与死锁预防）

题目种类：基础功能

推荐知识点：故障恢复概述、基于 REDO/UNDO 日志的恢复算法、恢复算法 ARIES

题目描述：

本题目要求需要实现日志管理器，在系统运行过程中把读写操作的日志通过日志缓冲区写入磁盘中，对于日志管理，参赛队伍需要实现 WAL 算法和 redo/undo 日志，并通过 redo/undo 日志对系统进行故障恢复，让数据库系统恢复到一致性状态。

在完成本任务之前，参赛队伍可以首先阅读项目结构文档中系统故障恢复的相关说明，以及代码框架中 src/recovery 文件夹下的代码文件。

提示：

（1）功能题目对代码框架没有限制，参赛队伍可以修改、增添、删除数据结构及接口，也可以对框架进行重构。

（2）在本题目中，测试场景为系统故障，且不包含故障恢复过程中的再次故障。

（3）本题目不包含检查点测试，每一次从日志文件的第一条记录进行故障恢复。

（4）对于索引操作，应记录物理日志而不是逻辑日志，防止索引出现不一致状态。

（5）本题目中包含两表 join 和 order by 操作。

测试示例：

在本测试中，系统会接收到若干事务，在运行过程中会接收到终止信号终止系统运行，当再次重启系统后，系统需要进行故障恢复，让数据库系统恢复到一致性状态。示例如下：

```
create table t1 (id int, num int);
```

```
begin;
```

```
insert into t1 values(1, 1);
```

```
commit;
```

```
begin;
```

```
insert into t1 values(2, 2);
```

```
crash // 系统接收到终止信号
```

```
...
```

```
重启系统
```

```
select * from t1;
```

测试输出要求：

本题目的测试分为单线程测试和多线程测试，单线程测试的输出通过数据库文件夹下的 output.txt 文件来进行正确性判断，多线程测试的输出通过客户端的返回字符串来进行正确性判断，两种测试方法的格式及输出文件要求参考其他题目的测试输出要求。